

Module 3 — Stretch: SQL Murder Mystery Investigation

Designed by [Diala Abedalqader](#)

Already submitted your PR in Slack? Please also paste your PR URL into TalentLMS → Module 3 → Tuesday Stretch so you receive credit. All stretch assignments are graded through TalentLMS for Honors designation.

Honors Track. This stretch assignment is not required for program completion but counts toward Honors distinction. Only work on this if you have completed all core assignments for this module and are On Track or Advanced. If you are behind on core work, focus there first.

The Challenge

A murder was committed in SQL City on January 15, 2018. The detective's database is full of clues — but the queries are slow. Your job isn't just to run queries; it's to understand **why they're slow** and **make them faster**.

You'll use `EXPLAIN ANALYZE` to read execution plans, add indexes, and measure the impact. This is a professional database skill — understanding why a query is slow and how to fix it is the difference between a working prototype and a production-ready system.

What You'll Learn

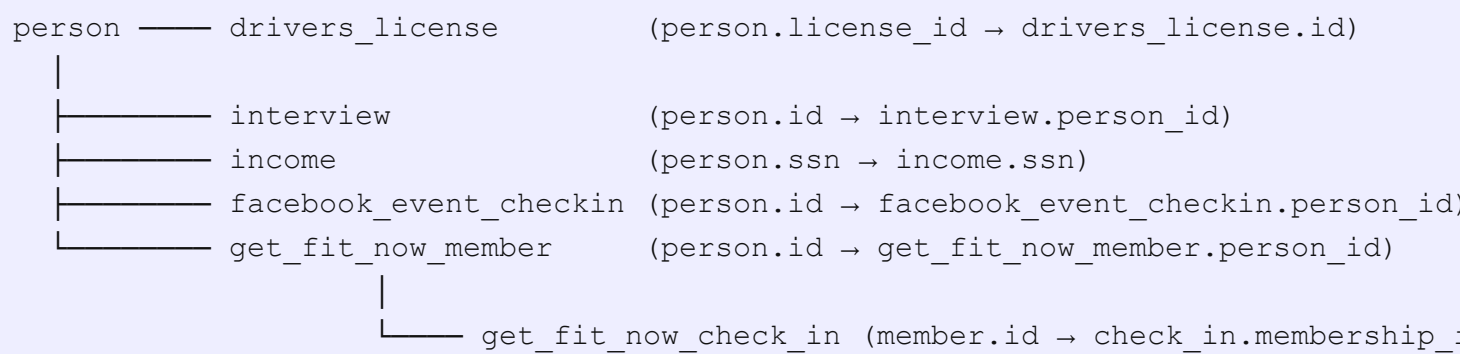
- Reading PostgreSQL query execution plans (`EXPLAIN ANALYZE`)
- Index types and when they help (B-tree for equality/range, partial indexes)
- The read/write tradeoff of indexing
- Making evidence-based optimization decisions

The Database

The database is named `murder_mystery` and runs in a PostgreSQL container.

Table	Rows	Description
<code>person</code>	10,011	People in the city (name, address, SSN, license)
<code>drivers_license</code>	10,007	Driver's license details (age, height, eye color, car info)
<code>crime_scene_report</code>	1,228	Police reports (date, type, description, city)
<code>interview</code>	4,991	Witness and suspect interview transcripts
<code>facebook_event_checkin</code>	20,011	Event attendance records
<code>get_fit_now_member</code>	184	Gym membership records
<code>get_fit_now_check_in</code>	2,703	Gym check-in/check-out times
<code>income</code>	7,514	Annual income per person (via SSN)

Schema Diagram



Setup

Clone the repo: <https://github.com/MsDiala/SQL-Murder-Mystery>

Start the database:

```
docker-compose up -d
```

Load the schema and data:

```
docker exec -i murder_db psql -U postgres -d murder_mystery < setup.sql
```

Connect to the database:

```
docker exec -it murder_db psql -U postgres -d murder_mystery
```

Verify the setup by checking that you see 8 tables with data:

```
SELECT 'crime_scene_report' AS table_name, COUNT(*) FROM crime_scene_report
UNION ALL SELECT 'drivers_license', COUNT(*) FROM drivers_license
UNION ALL SELECT 'facebook_event_checkin', COUNT(*) FROM facebook_event_checkin
UNION ALL SELECT 'get_fit_now_check_in', COUNT(*) FROM get_fit_now_check_in
UNION ALL SELECT 'get_fit_now_member', COUNT(*) FROM get_fit_now_member
UNION ALL SELECT 'income', COUNT(*) FROM income
UNION ALL SELECT 'interview', COUNT(*) FROM interview
UNION ALL SELECT 'person', COUNT(*) FROM person;
```

Tasks

1. Baseline Execution Plans

Run `EXPLAIN ANALYZE` on each of the 8 queries below. Save all output to `explain_baseline.md`.

For each query, record:

- Execution time (ms)
- Scan type (Seq Scan, Index Scan, Bitmap Index Scan)
- Join method (Nested Loop, Hash Join, Merge Join)
- Flag any Seq Scan on large tables — those are your optimization targets

Q1 — All murders in SQL City

```
EXPLAIN ANALYZE
SELECT date, description
FROM crime_scene_report
WHERE city = 'SQL City'
      AND type = 'murder'
ORDER BY date DESC;
```

Q2 — People with their driver's license details

```
EXPLAIN ANALYZE
SELECT p.name, p.address_number, p.address_street_name,
       dl.age, dl.eye_color, dl.hair_color, dl.car_make, dl.car_model
FROM person p
JOIN drivers_license dl ON p.license_id = dl.id
ORDER BY p.name;
```

Q3 — Gym members who checked in on January 9, 2018

```
EXPLAIN ANALYZE
SELECT m.name, m.membership_status, ci.check_in_time, ci.check_out_time
FROM get_fit_now_member m
JOIN get_fit_now_check_in ci ON m.id = ci.membership_id
WHERE ci.check_in_date = 20180109
ORDER BY ci.check_in_time;
```

Q4 — Gold gym members and their income

```
EXPLAIN ANALYZE
SELECT m.name, m.membership_status, i.annual_income
FROM get_fit_now_member m
JOIN person p ON m.person_id = p.id
JOIN income i ON p.ssn = i.ssn
WHERE m.membership_status = 'gold'
ORDER BY i.annual_income DESC;
```

Q5 — People who attended Facebook events in 2018

```
EXPLAIN ANALYZE
SELECT p.name, fe.event_name, fe.date
FROM person p
JOIN facebook_event_checkin fe ON p.id = fe.person_id
WHERE fe.date BETWEEN 20180101 AND 20181231
ORDER BY fe.date DESC;
```

Q6 — Red-haired Tesla drivers

```
EXPLAIN ANALYZE
SELECT p.name, dl.hair_color, dl.car_make, dl.car_model, dl.plate_number
FROM person p
JOIN drivers_license dl ON p.license_id = dl.id
WHERE dl.hair_color = 'red'
      AND dl.car_make = 'Tesla'
ORDER BY p.name;
```

Q7 — Interview transcripts mentioning the gym or murder

```
EXPLAIN ANALYZE
SELECT p.name, i.transcript
FROM interview i
JOIN person p ON i.person_id = p.id
WHERE i.transcript ILIKE '%gym%'
      OR i.transcript ILIKE '%murder%';
```

Q8 — Average income by car make

```
EXPLAIN ANALYZE
SELECT dl.car_make,
       COUNT(*) AS drivers,
       ROUND(AVG(i.annual_income), 0) AS avg_income,
       MIN(i.annual_income) AS min_income,
       MAX(i.annual_income) AS max_income
FROM drivers_license dl
JOIN person p ON dl.id = p.license_id
JOIN income i ON p.ssn = i.ssn
GROUP BY dl.car_make
ORDER BY avg_income DESC;
```

2. Add Indexes

Based on your execution plans, identify tables being fully scanned unnecessarily. Write your `CREATE INDEX` statements in `indexes.sql`, then run:

```
docker exec -i murder_db psql -U postgres -d murder_mystery < indexes.sql
```

Starter suggestions (add more based on your findings):

```
CREATE INDEX idx_crime_city_type ON crime_scene_report(city, type);
CREATE INDEX idx_person_license ON person(license_id);
CREATE INDEX idx_checkin_date ON get_fit_now_check_in(check_in_date);
CREATE INDEX idx_facebook_date ON facebook_event_checkin(date);
CREATE INDEX idx_facebook_person ON facebook_event_checkin(person_id);
```

3. Compare Performance

Re-run `EXPLAIN ANALYZE` on the same 8 queries after adding indexes. Save output to `explain_indexed.md`.

Note the execution time before and after for each query, and observe whether the scan type changed from Seq Scan to Index Scan.

4. Write a Report

Complete `performance_report.md` documenting:

- Which queries improved the most (and why the index helped)
- Which queries showed no improvement (e.g., small table, `ILIKE '%...'` wildcard, planner chose seq scan)
- The tradeoffs: faster reads vs. slower writes, additional storage
- Your production recommendation: which indexes would you actually keep?

Repository Structure

```
├─ README.md
├─ docker-compose.yml      ─ spins up PostgreSQL
├─ setup.sql               ─ schema + full data load
├─ indexes.sql             ─ your CREATE INDEX statements (edit this)
├─ explain_baseline.md     ─ Task 1 output (fill this in)
├─ explain_indexed.md      ─ Task 3 output (fill this in)
└─ performance_report.md   ─ Task 4 report (fill this in)
```

Resources

- [PostgreSQL: EXPLAIN](#)
- [PostgreSQL: Using EXPLAIN](#)
- [PostgreSQL: Indexes](#)
- [PostgreSQL: Index Types](#)
- [Original SQL Murder Mystery Game](#) — try solving the case too!

Submit

1. Create a branch (e.g., `stretch-performance`)
2. Add your completed files
3. Open a PR to `main`
4. Paste your PR URL into TalentLMS → Module 3 → Tuesday Stretch to submit this assignment